

High-Level Design (HLD): Premium Online Judge Platform

1. Problem Statement

The goal is to build an online judge platform similar to LeetCode and CodeChef, providing an environment where developers can solve coding challenges, run code against test cases, and submit their solutions for scoring. The platform needs to be highly interactive, robust, and feature a premium, visually stunning dark-mode user interface with micro-animations and seamless navigation.

2. Overview

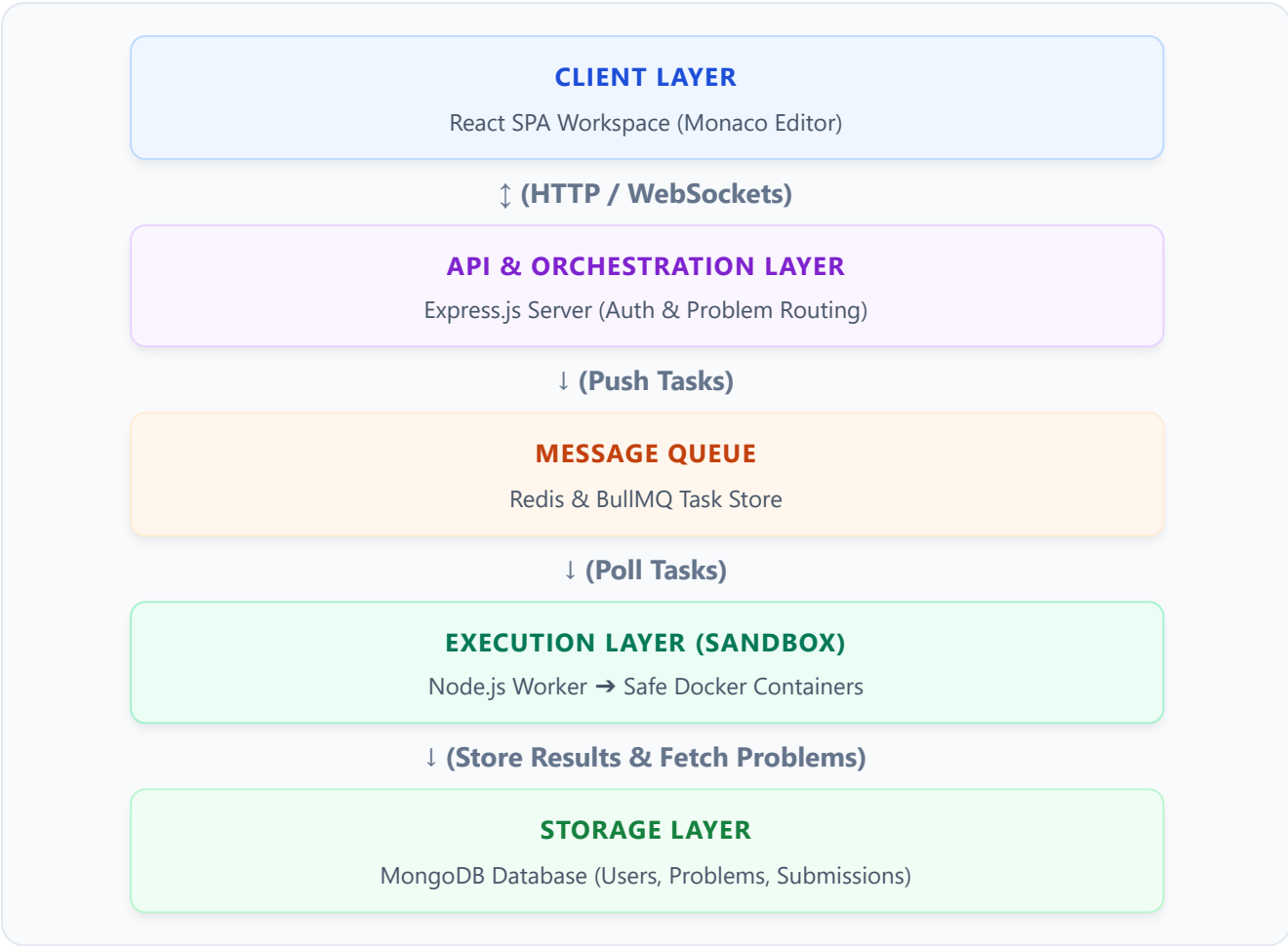
The platform uses a modern web architecture: - **Frontend**: A React Single Page Application (SPA) built using Vite for fast compilation, with styling driven by a custom Vanilla CSS design system. It embeds the Monaco Editor to provide an IDE-like coding workspace. - **Backend**: A Node.js and Express.js API gateway that manages users, problem sets, and submissions, and orchestrates code execution tasks. - **Database**: MongoDB handles storage of user profiles, problem statements, test cases, and submission histories. - **Execution Sandbox**: Runs submitted code securely using isolated Docker containers to prevent unauthorized system access, resource hogging, or malicious activity.

3. Key Features

- **Interactive Dashboard**: Circular progress charts for solved problems, a GitHub-style activity contribution calendar, and personalized metrics.
 - **IDE-Like Workspace**: Resizable split-pane layout containing the problem description, test case inputs, output console, and code editor.
 - **Real-Time Judging**: Asynchronous submission queue that updates the UI dynamically using WebSockets once evaluation is finished.
 - **Global Leaderboard**: Live rankings based on total points and submission performance metrics.
-

4. System Architecture

To ensure high availability, security, and scalability (especially during compilation and execution of untrusted user code), we use an **asynchronous microservices/worker-based architecture**.



System Components & Layout

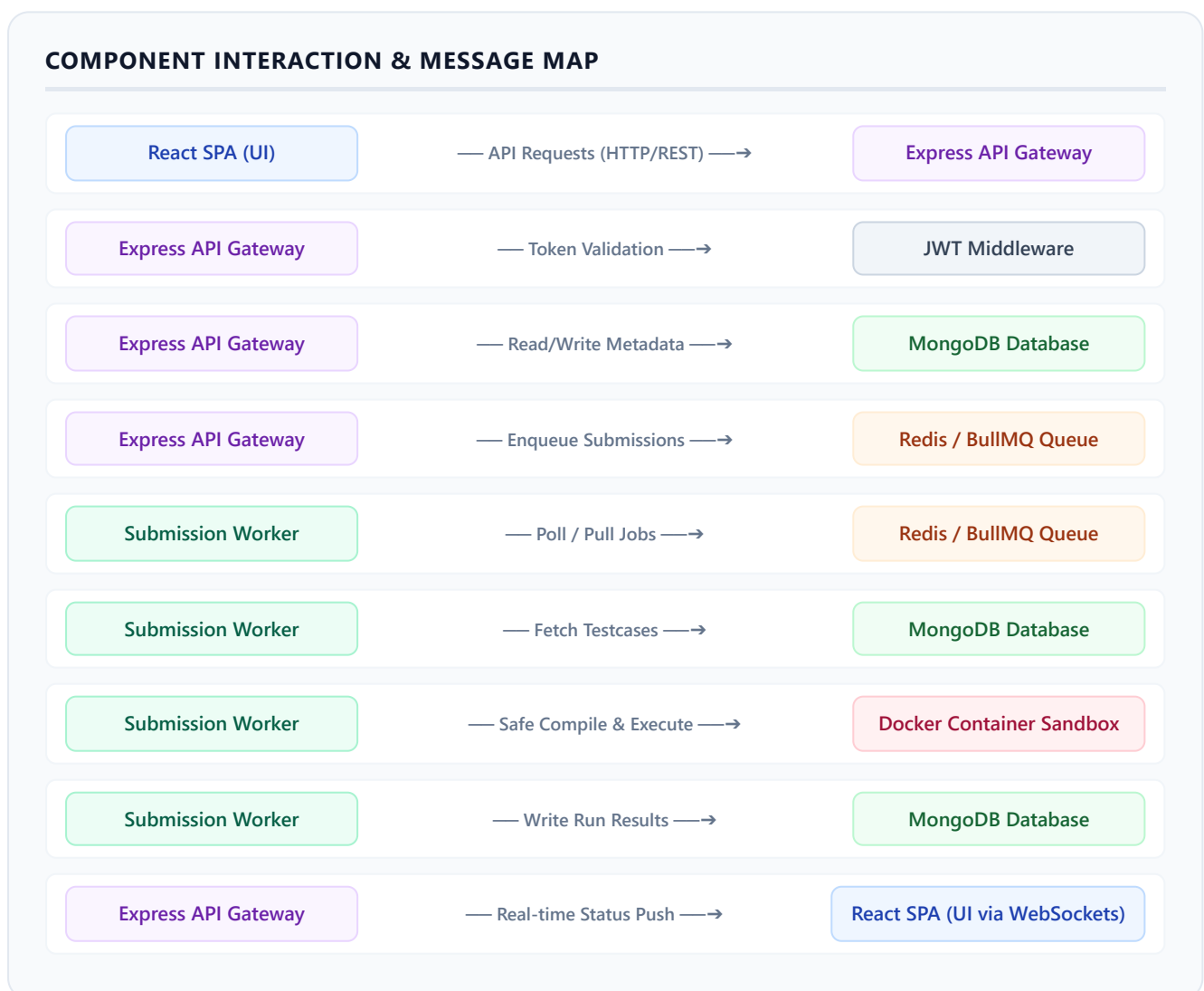
Component Layer	Technology Stack	Purpose / Action
Client Layer	React SPA (Vite, Vanilla CSS)	Premium user interface, dashboard metrics, and code editing workspace.
API & Gateway Layer	Node.js + Express.js	Manages routing, authentication (JWT), and problem/submission CRUD.
Message Queue	Redis + BullMQ	Acts as a buffer to hold code execution tasks asynchronously.
Execution Layer	Node.js Worker + Docker Sandbox	Pulls tasks from the queue, runs code inside isolated containers, and validates output.
Storage Layer	MongoDB + Redis Cache	Stores persistent application data (problems, submissions, users).

Key Architectural Concepts

1. **Decoupled Code Execution:** The API Gateway does not compile or run code. It offloads submissions to a message queue (Redis/BullMQ) and immediately returns a queuing status to the user. This prevents the server from freezing and protects it from Denial of Service (DoS) attacks.
2. **Dedicated Workers:** Isolated execution workers poll the submission queue, set up the execution environment, run the code against test cases in sandboxed Docker containers, and write results back to the database.
3. **Real-time Notifications:** WebSockets notify the React frontend when the worker completes execution, allowing the UI to transition from "Judging..." to "Accepted" or "Wrong Answer" instantly.

5. Component Interactions & Communication

To explain how components talk to each other, the interaction flow is structured as follows:



- **React Client & Express Gateway:** The frontend communicates with the backend via REST APIs for operations such as logging in, creating problems, fetching lists, or submitting solutions. It also establishes a persistent WebSocket connection to receive instant job execution results.
 - **Express Gateway & JWT Auth:** All requests are routed through a JSON Web Token middleware to parse credentials and authorize request context before hitting business routes.
 - **Express Gateway & Database:** Metadata, user profiles, dashboard metrics, and problem lists are read and written to MongoDB using Mongoose models.
 - **Express Gateway & Redis/BullMQ Queue:** Submissions are pushed as serializable job objects to the Redis task queue to offload heavy compilation CPU resources from the gateway.
 - **Worker & Redis Queue:** The Node.js Worker polls the queue. It takes one code job at a time to execute sequentially or concurrently depending on server core capacity.
 - **Worker & Database:** The Worker queries MongoDB to fetch specific test case files (inputs/expected outputs) for validation, and writes final status, run time, and memory metrics back to the database when complete.
 - **Worker & Docker Sandbox:** The Worker spawns a temporary Docker container, maps code as a read-only volume, executes the code inside the container under CPU/RAM constraints, and captures stdout and stderr.
-

6. Component Breakdown & Descriptions

A. Frontend (React SPA)

Built using React, Vite, and Vanilla CSS with a glassmorphism and modern dark-themed aesthetic. - **Problem Workspace:** A dual-pane IDE environment. - Left Panel: Tabbed navigation showing *Description*, *Submissions*, and *Leaderboard*. - Right Panel: Integrated **Monaco Editor** (the same editor that powers VS Code) with auto-completion, bracket matching, syntax highlighting, theme options, and a collapsible run console. - **User Dashboard:** Dynamic progress trackers showing easy/medium/hard problems solved using SVGs/Canvas charts, recent activities, user ranking, and personal metrics. - **Leaderboard:** Real-time ranking of users based on solved problems, submission speeds, and weekly contest performance.

B. Backend API Gateway (Node.js & Express.js)

Handles client requests, routes, business logic, and authentication. - `/api/auth`: Signup, Login (JWT tokens), OAuth (GitHub/Google). - `/api/problems`: CRUD operations for problems (Admin only for CUD), fetch lists, filters (tags, difficulty), and single problem details. - `/api/submissions`: Create submission (push to queue), fetch submissions for a

user/problem, and get live status. - `/api/dashboard` : Fetch analytics, user statistics, and leaderboards.

C. Message Queue (Redis & BullMQ)

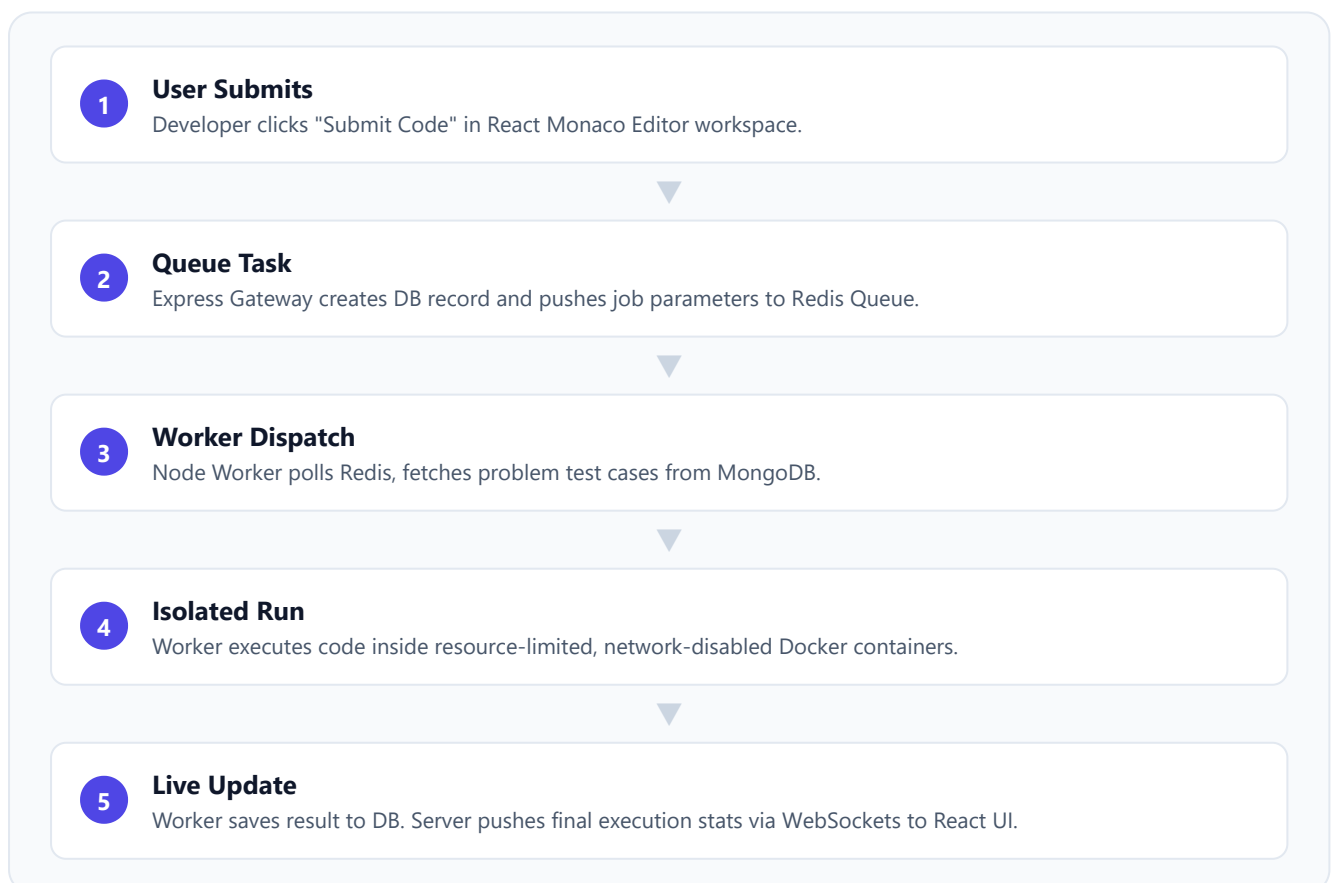
Acts as the buffer between the API Gateway and the Execution Worker. - Ensures no submission is lost. - Allows scaling the number of workers horizontally if traffic spikes.

D. Secure Judge Worker (Docker Sandbox)

The core compiler and runner. It does the following for each submission: 1. Fetches test cases (inputs and expected outputs) from the database or cache. 2. Writes user code to a temporary file. 3. Spawns an isolated Docker container with strict limits: - **Time Limit**: (e.g., 2 seconds) enforced using container timeouts. - **Memory Limit**: (e.g., 256MB) enforced via Docker memory constraints. - **Network Block**: No internet access within the container to prevent malicious data leaks. - **Process Limit (Fork Bomb Protection)**: Strictly limited process forks. 4. Executes the code against input files, capturing stdout, stderr, execution time, and memory usage. 5. Standardizes results: - **AC** (Accepted) - **WA** (Wrong Answer) - **TLE** (Time Limit Exceeded) - **MLE** (Memory Limit Exceeded) - **RE** (Runtime Error) - **CE** (Compilation Error)

7. Detailed Step-by-Step Submission Flow

The following sequence details how code execution is handled from the user interface:



1. **User Action:** The user writes a solution in the Monaco Editor and clicks **Submit**.
 2. **API Request:** The React SPA sends a `POST` request to `/api/submissions` with the problem ID, code, and language.
 3. **Queue Ingestion:** The Express server creates a submission record in MongoDB marked as `Queued` and pushes the job parameters (submission ID, code, language) to the Redis/BullMQ queue.
 4. **Immediate Acknowledgment:** The Express server returns the submission ID and status to the client, which displays a loading/judging spinner.
 5. **Worker Polling:** A background execution worker pulls the job from Redis and queries MongoDB to fetch the corresponding test cases (inputs/outputs) for the problem.
 6. **Sandboxed Run:** For each test case, the worker spawns an isolated Docker container, writes the user code inside, runs it with the input parameters, and measures execution time and memory.
 7. **Result Persistence:** The worker compiles the test case outcomes, updates the submission status in MongoDB (e.g. `Accepted`, `Wrong Answer`, `Time Limit Exceeded`), and fires a status update to the gateway.
 8. **Real-time Push:** The Express server sends a WebSocket event containing the results to the client.
 9. **UI Update:** The React SPA updates the screen, replacing the judging spinner with the final run results (execution time, memory used, test case results).
-

8. Technology Stack Justification

Layer	Technology	Rationale & How it is Built
Frontend	React & Vite	Extremely fast hot module reloading, component-based development, large ecosystem. Custom styling via Vanilla CSS modules utilizing modern variables, CSS grid/flexbox, transitions, and glassmorphic designs to create a premium IDE look.
Code Editor	Monaco Editor	The editor behind VS Code. Provides a premium developer experience with code autocompletion, language switching, theme selection (Dark, Light, High Contrast), and multi-cursor editing.
Backend	Node.js & Express	Single language (JavaScript/TypeScript) end-to-end. High concurrency, asynchronous I/O ideal for handling API routes and WebSocket events without blocking.
Database	MongoDB & Mongoose	Schema-flexibility works wonders for storing problem statements, markdown text, test cases (variable structure), and submissions logs. Mongoose provides object modeling and validation.
Queue	Redis & BullMQ	Redis provides lightning-fast memory storage. BullMQ manages job prioritization, retry logic, concurrency control, and delayed tasks.
Sandbox	Docker	Isolates the running code from the host. Enforces resource quotas (CPU, RAM) and shuts down malicious activity (e.g., trying to read system files or run fork bombs).

9. Database Schema Designs

User Schema (users)

- `_id` : ObjectId
- `username` : String (Unique)
- `email` : String (Unique)
- `passwordHash` : String

- `role` : String (Enum: 'user' , 'admin')
- `solvedCount` : Number
- `profile` : { avatarUrl, bio, website }
- `createdAt` : Date

Problem Schema (`problems`)

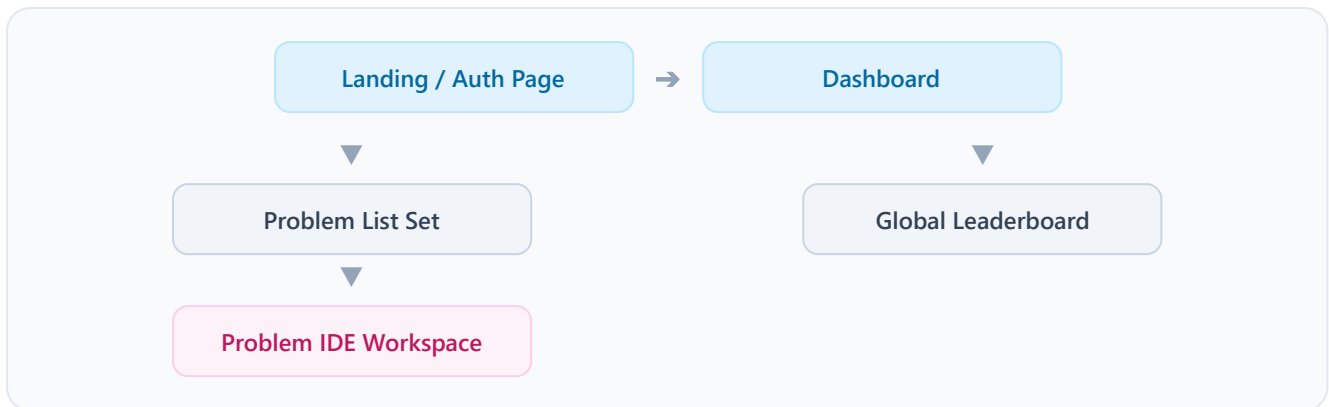
- `_id` : ObjectId
- `title` : String
- `slug` : String (Unique, e.g. 'two-sum')
- `description` : String (Markdown content)
- `difficulty` : String (Enum: 'Easy' , 'Medium' , 'Hard')
- `tags` : [String]
- `timeLimit` : Number (in ms)
- `memoryLimit` : Number (in MB)
- `testCases` : [{ `input` : String, `expectedOutput` : String, `isSample` : Boolean (true shows it on problem description) }]
- `createdAt` : Date

Submission Schema (`submissions`)

- `_id` : ObjectId
- `userId` : ObjectId (Ref: User)
- `problemId` : ObjectId (Ref: Problem)
- `code` : String
- `language` : String (Enum: 'cpp' , 'java' , 'python' , 'javascript')
- `status` : String (Enum: 'Queued' , 'Judging' , 'Accepted' , 'Wrong Answer' , 'Time Limit Exceeded' , 'Memory Limit Exceeded' , 'Runtime Error' , 'Compilation Error')
- `runtime` : Number (ms)
- `memory` : Number (KB)
- `errorMessage` : String (for compilation or runtime errors)
- `createdAt` : Date

10. Premium UI Page Navigation & Layout

We design a highly interactive, gorgeous visual layout with dark mode glassmorphism (`backdrop-filter: blur()`).



UI Navigation Map

- **Landing Page** (Auth Gateway)
- → **Dashboard** (Circular charts, GitHub-style calendar, recent activities)
- → **Problem Set List** (Difficulty filter, tag search, completion badges)
 - → **Problem Solve Workspace (IDE)**
 - Left Pane: Problem description, sample test cases, personal submissions
 - Right Pane: Monaco Editor (with themes & language pickers), Run/Submit console
- → **Global Leaderboard** (Rankings, scores, rating indicators)

Key Screens

1. Dashboard

2. **Header:** Sticky glass panel, blurred background, search bar, profile avatar drop-down.

3. **Main Panel:** Grid of cards.

- Card 1: *Progress Doughnut Chart* showing solved ratios (green for easy, amber for medium, ruby red for hard).
- Card 2: *Activity Calendar Heatmap* (GitHub-style submission grid) showing frequency of submissions.
- Card 3: *Recent Submissions Table* with live-status badges.
- Card 4: *Recommended Problems* list matching user's weak topics.

4. Problem Workspace (The "IDE" View)

5. Header: Back button to problem list, toggle button for Fullscreen, Run / Submit buttons.

6. Split Layout (Resizable panes):

- **Left Side (tabs):**
- *Description:* HTML rendering of Markdown problem statements, sample test cases with copy buttons, constraints block.

- *Submissions*: History of previous attempts, including execution speed, memory used, and codes.
- **Right Side (tabs)**:
 - *Editor Pane*: Full-screen Monaco Editor window. Language picker floating top-right.
 - *Console Pane* (slide-up): Testcases input editor and stdout console output.

7. Leaderboard

- 8. Live ranks of top performers with colorful badges (Gold, Silver, Bronze icons).
- 9. Metrics: Rank, Name, Total Score, Problems Solved, and Contest Rating.

11. Plan for Sandbox Execution Setup

For a local setup and development ease: - We can construct a simple Dockerfile for a generic runner. - The worker executes: `docker run --rm -v C:\temp\workspace:/app -w /app --network none --memory 256m --memory-swap 256m my-compiler-image python execute.py` - We will map submission directories into the Docker container as read-only volumes to allow code read, preventing files from being permanently modified. - For Javascript, Python, C++, and Java, standard compilers are pre-installed in the Docker image.

Done by: Harshith